SPORF: Sparse Projection Oblique Random Forests

1 Overview

Brief outline of the SPORF algorithm and notation. An ensemble is constructed from a set of single trees.

Algorithm 1: SPORF Tree Growth — high-level pseudocode

Input: dataset $D \in \mathbb{R}^{n \times d} \times \{\mathbb{R}^n | \mathbb{Z}^n\}$, number of trees T, bootstrap ratio b, projections per split P, sparse random vector density N_{NZ}, max quantile bins N_Q

Draw bootstrap sample D_t from D with ratio b;

Divide D_t into structure sample $D_{t,s,0}$ and honest sample $D_{t,h,0}$;

Initialize node queue with root node $N_{t,0}(D_{t,s,0}, D_{t,h,0})$;

while *node queue not empty* **do**

- Pop node $N_{t,i}(D_{t,s,i}, D_{t,h,i})$ from queue;
- Check stopping criterion (e.g., max depth, min samples, purity) and continue if not met;
- Generate P random projections $\{\mathbf{w}_p\}_{p=1}^{P}$ with number of nonzero coefficients $\sim \text{Bin}(d, N_{NZ}/d)$ and values $\sim \text{Unif}(\{-1, 0, +1\})$;
- Project data: $\mathbf{z}_p = \mathbf{w}_p^\top \mathbf{x}$ for all p and $\mathbf{x} \in D_{t,s,i}$;
- Sample $\min(N_Q, |\mathbf{z}_p|)$ quantile cutoff indices for each projection $\mathbf{w}_p$, $\sim \text{Unif}([0, \ldots, |\mathbf{z}_p|])$;
- Compute quantile counts for each projection $\mathbf{w}_p$ and cutoff index to accelerate split evaluation;
- For each projection $\mathbf{w}_p$, find best split $\theta_p^* = \arg\max_{\theta_{p,q}} \text{Gain}(D_{t,s,i}, \theta_{p,q})$ on projected features $\mathbf{w}_p^\top \mathbf{x}$, where Gain is the reduction in chosen loss (e.g., Gini, MSE) and q denotes quantile cutoff indices;
- Select best split $\theta^* = \arg\max_p \text{Gain}(D_{t,s,i}, \theta_p^*)$;
- Store split parameters θ^* and projection $\mathbf{w}^*$ in node $N_{t,i}$;
- Partition data into left/right subsets based on θ^* and add child nodes $N_{t,L}(D_{t,s,i}^{left}, D_{t,h,i}^{left})$ and $N_{t,R}(D_{t,s,i}^{right}, D_{t,h,i}^{right})$ to queue, where L and R denote respective left and right child node indices;

Output: tree τ_t